

```
#####
#
# COURS : TOUTES LES BRANCHES DE LA PHYSIQUE
# Théorie · Démonstrations · Lois · Code Python natif
#
#####
#
# COMMENT LIRE CE COURS ?
#
# [THÉORIE]      – L'idée physique, pourquoi ça existe
# [DÉFINITION]   – La loi ou formule formelle
# [DÉMONSTRATION] – Dérivation ou raisonnement pas à pas
# [EXEMPLE]      – Cas concret chiffré
# [CODE PYTHON]  – Programme qui simule ou calcule
# [PIÈGE]        – Erreur classique à éviter
#
# Fil directeur :
# La physique décrit le monde réel avec des mathématiques.
# Chaque branche répond à une question fondamentale :
# Mécanique      → Comment les objets bougent-ils ?
# Thermodynamique → Comment l'énergie se transforme-t-elle ?
# Électromagnétisme → Comment interagissent charges et champs ?
# Optique        → Comment se comporte la lumière ?
# Ondes          → Comment se propage une perturbation ?
# Mécanique quantique → Que se passe-t-il à l'échelle atomique ?
# Relativité     → Que se passe-t-il à grande vitesse / masse ?
# Physique statistique → Comment relier micro et macro ?
# Physique nucléaire → Qu'y a-t-il dans le noyau ?
# Physique des fluides → Comment coule un fluide ?
#
#####
```

```
┌
├
||  BRANCHE 1 – MÉCANIQUE CLASSIQUE  ||
└
└
```

## 1.1 CINÉMATIQUE – décrire le mouvement

### [THÉORIE]

La cinématique décrit COMMENT un objet se déplace, sans se soucier de POURQUOI (les causes viendront avec la dynamique).

Trois grandeurs fondamentales : position  $x(t)$ , vitesse  $v(t)$ , accélération  $a(t)$ . Elles sont liées par la dérivation (ou intégration).

### [DÉFINITION]

Position :  $x(t)$  [mètre, m]

Vitesse :  $v(t) = dx/dt$  [m/s]  
Accélération :  $a(t) = dv/dt = d^2x/dt^2$  [m/s<sup>2</sup>]

Mouvement rectiligne uniformément accéléré (MRUA) – accélération constante  $a$  :

$$\begin{aligned}v(t) &= v_0 + a \cdot t \\x(t) &= x_0 + v_0 \cdot t + (1/2) \cdot a \cdot t^2 \\v^2 &= v_0^2 + 2 \cdot a \cdot (x - x_0)\end{aligned}$$

Chute libre ( $a = g = 9.81 \text{ m/s}^2$ , vers le bas) :

$$y(t) = y_0 + v_0 \cdot t - (1/2) \cdot g \cdot t^2$$

Mouvement circulaire uniforme (rayon  $R$ , vitesse angulaire  $\omega$ ) :

$$\begin{aligned}v &= \omega \cdot R && [\text{m/s}] \\a_c &= v^2/R = \omega^2 \cdot R && (\text{accélération centripète, vers le centre}) \\T &= 2\pi / \omega && [\text{période, secondes}]\end{aligned}$$

[DÉMONSTRATION] Équations du MRUA depuis  $a = \text{constante}$

$a(t) = a = \text{constante}$   
Intégration :  $v(t) = \int a \, dt = a \cdot t + C_1 \rightarrow \text{avec } v(0) = v_0 : v(t) = v_0 + a \cdot t$   
Intégration :  $x(t) = \int v \, dt = v_0 \cdot t + (1/2) \cdot a \cdot t^2 + C_2$   
avec  $x(0) = x_0 : x(t) = x_0 + v_0 \cdot t + (1/2) \cdot a \cdot t^2$   
Élimination de  $t$  entre les deux :  
 $t = (v - v_0)/a \rightarrow x - x_0 = v_0 \cdot (v - v_0)/a + (1/2) \cdot a \cdot (v - v_0)^2/a^2$   
 $= (v^2 - v_0^2)/(2a)$   
Donc  $v^2 = v_0^2 + 2 \cdot a \cdot (x - x_0) \quad \square$

[EXEMPLE]

Balle lancée vers le haut à  $v_0 = 20 \text{ m/s}$  depuis le sol :  
 $t_{\text{max}} = v_0/g = 20/9.81 \approx 2.04 \text{ s}$  (à la hauteur max,  $v = 0$ )  
 $y_{\text{max}} = v_0^2/(2g) = 400/19.62 \approx 20.4 \text{ m}$   
 $t_{\text{retour}} = 2 \cdot t_{\text{max}} \approx 4.08 \text{ s}$

[CODE PYTHON]

---

```
import math

g = 9.81 # m/s²

def cinematique_mrue(x0, v0, a, t):
    v = v0 + a * t
    x = x0 + v0 * t + 0.5 * a * t**2
    return x, v

def chute_libre(y0, v0, t):
    return cinematique_mrue(y0, v0, -g, t)

# Simulation balle lancée vers le haut
y0, v0 = 0, 20 # m, m/s
print("Simulation balle lancée à v0=20 m/s :")
print(f"{'t (s)':>6} | {'y (m)':>8} | {'v (m/s)':>8}")
print("-" * 30)
for t_ms in range(0, 450, 50):
    t = t_ms / 100
```

```

y, v = chute_libre(y0, v0, t)
if y >= -0.01:
    print(f"{t:>6.2f} | {y:>8.3f} | {v:>8.3f}")

t_max = v0 / g
y_max = v0**2 / (2 * g)
print(f"\nHauteur maximale : {y_max:.2f} m à t = {t_max:.2f} s")
print(f"Retour au sol      : t = {2*t_max:.2f} s")

# Mouvement circulaire
R = 5.0      # m
omega = 2.0  # rad/s
v_tang = omega * R
a_centripete = omega**2 * R
T = 2 * math.pi / omega
print(f"\nCirculaire : R={R}m, omega={omega} rad/s")
print(f"  v = {v_tang} m/s | a_c = {a_centripete} m/s² | T = {T:.3f} s")

```

---



---

## 1.2 DYNAMIQUE – les lois de Newton

---

### [THÉORIE]

Newton a formulé trois lois qui expliquent POURQUOI les objets bougent.  
 Ce sont les fondements de toute la mécanique classique.  
 Valables pour des vitesses bien inférieures à  $c$  (la lumière).

### [DÉFINITION]

1re LOI (inertie) :

Tout corps persévère dans son état de repos ou de mouvement rectiligne uniforme tant qu'aucune force ne s'exerce sur lui.  
 $\Sigma(F) = 0 \Leftrightarrow a = 0$

2e LOI (fondamentale) :

$\Sigma(F) = m * a$   
 La somme des forces = masse × accélération.  
 [F en Newton N =  $\text{kg} \cdot \text{m/s}^2$ ]

3e LOI (action-réaction) :

$F_{AB} = -F_{BA}$   
 Si A exerce une force sur B, alors B exerce une force égale et opposée sur A.

GRAVITATION UNIVERSELLE (Newton) :

$F = G * m_1 * m_2 / r^2$   
 $G = 6.674e-11 \text{ N} \cdot \text{m}^2/\text{kg}^2$   
 Direction : attractive, le long de la droite reliant les deux masses.

POIDS :  $P = m * g$  ( $g = 9.81 \text{ m/s}^2$  à la surface terrestre)

### [DÉMONSTRATION] $g$ à partir de la gravitation universelle

La Terre (masse  $M_T$ , rayon  $R_T$ ) attire un objet de masse  $m$  :

```
F = G * MT * m / RT^2
```

Cette force = poids =  $m * g$

```
m * g = G * MT * m / RT^2
```

```
g = G * MT / RT^2
```

```
g = (6.674e-11 * 5.972e24) / (6.371e6)^2
```

```
g ≈ 9.81 m/s2    □
```

[EXEMPLE]

Bloc  $m = 5$  kg sur un plan incliné à  $30^\circ$ , sans frottement :

Force parallèle au plan :  $F_{\text{par}} = m * g * \sin(30^\circ) = 5 * 9.81 * 0.5 = 24.5$  N

Accélération :  $a = g * \sin(30^\circ) = 4.9$  m/s<sup>2</sup>

[CODE PYTHON]

---

```
G_const = 6.674e-11    # N·m2/kg2
```

```
g = 9.81                # m/s2
```

```
def gravitation(m1, m2, r):  
    """Force gravitationnelle entre deux masses"""  
    return G_const * m1 * m2 / r**2
```

```
def acceleration_plan_incline(angle_deg, mu=0):  
    """  
    Accélération sur plan incliné.  
    angle_deg : angle en degrés  
    mu        : coefficient de frottement cinétique  
    """  
  
    theta = math.radians(angle_deg)  
    return g * (math.sin(theta) - mu * math.cos(theta))
```

```
# Gravitation universelle
```

```
MT = 5.972e24    # kg Terre
```

```
RT = 6.371e6    # m    rayon Terre
```

```
g_calcule = G_const * MT / RT**2
```

```
print(f"g calculé depuis G et MT : {g_calcule:.4f} m/s2")
```

```
# Plan incliné
```

```
for angle in [10, 20, 30, 45]:  
    a = acceleration_plan_incline(angle)  
    print(f"Plan à {angle}° : a = {a:.3f} m/s2")
```

```
# Simulation Newton : chute avec frottement de l'air
```

```
def simulation_chute_frottement(m, k, v0, dt, t_max):  
    """  
    m : masse (kg)  
    k : coefficient de frottement (N·s/m)  
    v0 : vitesse initiale (m/s)  
    Modèle :  $F_{\text{net}} = m * g - k * v$  (vers le bas positif)  
    """  
  
    v = v0  
    y = 0  
    resultats = [(0, y, v)]  
    t = 0
```

```

while t < t_max:
    a = g - (k / m) * v
    v += a * dt
    y += v * dt
    t += dt
    resultats.append((round(t, 4), round(y, 4), round(v, 4)))
return resultats

res = simulation_chute_frottement(m=1, k=0.5, v0=0, dt=0.1, t_max=3)
print("\nChute avec frottement (m=1kg, k=0.5) :")
print(f"{'t':>5} | {'y':>8} | {'v':>8}")
for t, y, v in res[:5]:
    print(f"{'t':>5.1f} | {'y':>8.3f} | {'v':>8.3f}")

v_limite = 1 * g / 0.5
print(f"Vitesse limite théorique : {v_limite} m/s")

```

---



---

### 1.3 ÉNERGIE ET TRAVAIL

---

#### [DÉFINITION]

TRAVAIL d'une force  $F$  sur un déplacement  $d$  :

$$W = F \cdot d = F * d * \cos(\theta) \quad [\text{Joule, J}]$$

(produit scalaire force-déplacement)

#### ÉNERGIE CINÉTIQUE :

$$E_c = (1/2) * m * v^2 \quad [J]$$

#### ÉNERGIE POTENTIELLE DE PESANTEUR :

$$E_p = m * g * h \quad [J] \quad (h = \text{hauteur})$$

#### ÉNERGIE MÉCANIQUE :

$$E_m = E_c + E_p$$

#### THÉORÈME DE L'ÉNERGIE CINÉTIQUE :

$$W_{\text{total}} = \Delta(E_c) = E_{c_{\text{final}}} - E_{c_{\text{initial}}}$$

#### CONSERVATION DE L'ÉNERGIE MÉCANIQUE (sans frottement) :

$$E_m = \text{constante} \quad \rightarrow \quad E_c + E_p = \text{constante}$$

#### PUISSANCE :

$$P = W / t = F * v \quad [\text{Watt, } W = J/s]$$

#### [DÉMONSTRATION] Vitesse d'un objet au bas d'une pente (conservation énergie)

En haut :  $E_c = 0$  (départ du repos),  $E_p = m * g * h$

En bas :  $E_c = (1/2) * m * v^2$ ,  $E_p = 0$

Conservation :  $m * g * h = (1/2) * m * v^2$

$$v = \sqrt{2 * g * h} \quad (\text{indépendant de la masse !}) \quad \square$$

#### [CODE PYTHON]

---

```

def ec(m, v):
    return 0.5 * m * v**2

def ep(m, h, g_val=9.81):
    return m * g_val * h

def vitesse_depuis_hauteur(h, v0=0):
    return math.sqrt(v0**2 + 2 * g * h)

# Conservation de l'énergie
h_initial = 10 # m
m = 2 # kg
v_bas = vitesse_depuis_hauteur(h_initial)

print(f"Objet de {m}kg lâché depuis {h_initial}m :")
print(f"  Ep initiale = {ep(m, h_initial):.2f} J")
print(f"  v au bas = {v_bas:.3f} m/s")
print(f"  Ec au bas = {ec(m, v_bas):.2f} J")
print(f"  Vérification Ep_i = Ec_f : {abs(ep(m, h_initial) - ec(m, v_bas)) < 1e-9}")

# Simulation pendule (conservation énergie)
def simulation_pendule(L, theta0_deg, dt=0.01, t_max=4):
    """
    Pendule simple :  $d^2\theta/dt^2 = -(g/L)\sin(\theta)$ 
    Approximation petits angles :  $\sin(\theta) \approx \theta$ 
    """
    theta = math.radians(theta0_deg)
    omega = 0.0
    resultats = [(0, math.degrees(theta), omega)]
    t = 0
    while t < t_max:
        alpha = -(g / L) * math.sin(theta) # accélération angulaire
        omega += alpha * dt
        theta += omega * dt
        t += dt
        resultats.append((round(t, 3), round(math.degrees(theta), 4), round(omega, 4)))
    return resultats

L = 1.0 # m
T_theorique = 2 * math.pi * math.sqrt(L / g)
print(f"\nPendule L=1m : T théorique = {T_theorique:.4f} s")

res_pendule = simulation_pendule(L, theta0_deg=15, dt=0.005, t_max=2)
print(f"{'t':>6} | {'theta(°)':>10} | {'omega':>8}")
for row in res_pendule[:40]:
    print(f"{'row[0]:>6.3f} | {'row[1]:>10.4f} | {'row[2]:>8.4f}")

```

---

#### [DÉFINITION]

Quantité de mouvement (impulsion) :

$$p = m * v \quad [\text{kg}\cdot\text{m/s}]$$

2e loi de Newton :  $F = dp/dt$

CONSERVATION : si  $\Sigma(F_{\text{ext}}) = 0$ , alors  $p_{\text{total}} = \text{constante}$ .

CHOC ÉLASTIQUE (énergie cinétique conservée) :

$$m_1*v_1 + m_2*v_2 = m_1*v_1' + m_2*v_2' \quad (\text{quantité de mouvement})$$

$$E_{c1} + E_{c2} = E_{c1}' + E_{c2}' \quad (\text{énergie cinétique})$$

Solutions :

$$v_1' = ((m_1 - m_2)*v_1 + 2*m_2*v_2) / (m_1 + m_2)$$

$$v_2' = ((m_2 - m_1)*v_2 + 2*m_1*v_1) / (m_1 + m_2)$$

CHOC PARFAITEMENT INÉLASTIQUE (les deux corps fusionnent) :

$$v_{\text{finale}} = (m_1*v_1 + m_2*v_2) / (m_1 + m_2)$$

#### [CODE PYTHON]

---

```
def choc_elastique_1d(m1, v1, m2, v2):
    """Vitesses après choc élastique 1D"""
    v1p = ((m1 - m2)*v1 + 2*m2*v2) / (m1 + m2)
    v2p = ((m2 - m1)*v2 + 2*m1*v1) / (m1 + m2)
    return v1p, v2p

def choc_inelastique(m1, v1, m2, v2):
    return (m1*v1 + m2*v2) / (m1 + m2)

m1, v1 = 2.0, 5.0
m2, v2 = 1.0, -2.0

v1p, v2p = choc_elastique_1d(m1, v1, m2, v2)
print("Choc élastique :")
print(f"  Avant : p = {m1*v1+m2*v2:.2f} kg·m/s | Ec = {ec(m1,v1)+ec(m2,v2):.2f} J")
print(f"  Après : p = {m1*v1p+m2*v2p:.2f} kg·m/s | Ec = {ec(m1,v1p)+ec(m2,v2p):.2f} J")
print(f"  v1' = {v1p:.3f} m/s | v2' = {v2p:.3f} m/s")

vf = choc_inelastique(m1, v1, m2, v2)
print(f"\nChoc inélastique : v_finale = {vf:.3f} m/s")
print(f"  Énergie perdue = {ec(m1,v1)+ec(m2,v2) - ec(m1+m2, vf):.3f} J")
```

---

#### [PIÈGE]

La conservation de l'énergie mécanique ne s'applique qu'en l'absence de frottements et de chocs inélastiques.

Mais la conservation de la quantité de mouvement est TOUJOURS vraie si les forces extérieures sont nulles – même avec frottement.

---

## 2.1 TEMPÉRATURE, CHALEUR ET GAZ PARFAIT

---

### [THÉORIE]

La thermodynamique étudie les échanges d'énergie sous forme de chaleur et de travail. La "température" mesure l'agitation moyenne des atomes. Un gaz parfait est un modèle idéal : particules ponctuelles, sans interaction entre elles – valide pour les gaz rares à basse pression.

### [DÉFINITION]

Température :

Celsius → Kelvin :  $T(K) = T(^{\circ}C) + 273.15$

Le zéro absolu :  $0\text{ K} = -273.15^{\circ}C$  (agitation nulle)

LOI DES GAZ PARFAITS :

$$P * V = n * R * T$$

P : pression [Pascal, Pa]

V : volume [m<sup>3</sup>]

n : nombre de moles [mol]

R : constante des gaz parfaits = 8.314 J/(mol·K)

T : température [Kelvin]

CHALEUR MASSIQUE :

$$Q = m * c * \Delta T$$

c : capacité thermique massique [J/(kg·K)]

(eau : c = 4186 J/(kg·K))

TRANSFERTS THERMIQUES :

Conduction :  $Q/t = k * A * \Delta T / L$  (k : conductivité thermique)

Rayonnement :  $P = \epsilon * \sigma * A * T^4$  (Stefan-Boltzmann)

Convection :  $Q = h * A * \Delta T$

[DÉMONSTRATION] Pression d'un gaz en augmentant la température (V constant)

Loi des gaz parfaits à V constant (processus isochore) :

$$P_1/T_1 = P_2/T_2 \quad (\text{loi de Gay-Lussac})$$

Si T double : P double. L'agitation des particules augmente → plus de chocs sur les parois → pression plus élevée. □

### [CODE PYTHON]

---

```
R_gaz = 8.314 # J/(mol·K)
```

```
sigma_sb = 5.67e-8 # W/(m²·K⁴) Stefan-Boltzmann
```

```
def celsius_kelvin(T_C):  
    return T_C + 273.15
```

```
def loi_gaz_parfait(P=None, V=None, n=None, T=None):  
    """Résout PV = nRT pour la variable manquante (None)"""  
    if P is None:
```



```

    return n * R_gaz * T / V
elif V is None:
    return n * R_gaz * T / P
elif n is None:
    return P * V / (R_gaz * T)
elif T is None:
    return P * V / (n * R_gaz)

def chaleur(m, c, delta_T):
    return m * c * delta_T

# Gaz parfait
n = 1.0          # 1 mole
T = celsius_kelvin(25)  # 25°C → 298.15 K
V = 0.0224       # m³ (~22.4 L, volume molaire à 0°C)

P = loi_gaz_parfait(V=V, n=n, T=T)
print(f"Pression à 25°C, V=22.4L, 1 mol : {P:.1f} Pa ({P/101325:.3f} atm)")

# Loi de Gay-Lussac (V constant)
P1 = 101325 # Pa
T1 = celsius_kelvin(20)
T2 = celsius_kelvin(100)
P2 = P1 * T2 / T1
print(f"\nGay-Lussac : T: 20→100°C → P: {P1:.0f}→{P2:.0f} Pa")

# Chaleur pour chauffer 1L d'eau de 20°C à 100°C
m_eau = 1.0      # kg
c_eau = 4186     # J/(kg·K)
Q = chaleur(m_eau, c_eau, 80)
print(f"\nChauffer 1L eau (20→100°C) : Q = {Q:.0f} J = {Q/1000:.2f} kJ")

# Rayonnement du corps humain (T ≈ 37°C = 310 K)
T_corps = celsius_kelvin(37)
A_corps = 1.8    # m² (surface moyenne)
eps = 0.97       # émissivité peau
P_rayonne = eps * sigma_sb * A_corps * T_corps**4
print(f"\nRayonnement corps humain : {P_rayonne:.1f} W")

```

---



---

## 2.2 LES QUATRE LOIS DE LA THERMODYNAMIQUE

---

### [DÉFINITION]

LOI ZÉRO : Si A est en équilibre thermique avec C, et B aussi, alors A et B sont en équilibre thermique entre eux.  
(Fondement de la notion de température.)

1re LOI (conservation de l'énergie) :

$$\Delta U = Q + W$$

Variation d'énergie interne = chaleur reçue + travail reçu

L'énergie se conserve : on ne peut pas créer d'énergie.

[Pour un gaz parfait :  $\Delta U = n \cdot C_v \cdot \Delta T$ ]

2e LOI (entropie) :

L'entropie  $S$  d'un système isolé ne peut qu'augmenter.

$\Delta S \geq 0$  (égalité si réversible)

Conséquences : la chaleur va spontanément du chaud vers le froid.

Rendement d'une machine thermique :  $\eta \leq 1 - T_{\text{froid}}/T_{\text{chaud}}$

(Rendement de Carnot)

3e LOI : L'entropie d'un cristal parfait est nulle à 0 Kelvin.

[DÉMONSTRATION] Rendement de Carnot  $\eta = 1 - T_f/T_c$

Machine thermique entre source chaude  $T_c$  et source froide  $T_f$ .

Par la 1re loi :  $W = Q_c - Q_f$

Par la 2e loi (Clausius) :  $Q_f/T_f \geq Q_c/T_c \rightarrow Q_f \geq Q_c \cdot T_f/T_c$

Rendement :  $\eta = W/Q_c = 1 - Q_f/Q_c \leq 1 - T_f/T_c$

Maximum atteint (cycle réversible) :  $\eta_{\text{max}} = 1 - T_f/T_c$  □

[CODE PYTHON]

---

```
def rendement_carnot(T_chaud_C, T_froid_C):
    Tc = celsius_kelvin(T_chaud_C)
    Tf = celsius_kelvin(T_froid_C)
    return 1 - Tf / Tc

def entropie_echange(Q, T):
    """delta_S = Q/T (processus réversible)"""
    return Q / T

# Rendements de Carnot pour différentes machines
machines = [
    ("Moteur à vapeur", 600, 30),
    ("Centrale nucléaire", 300, 20),
    ("Moteur automobile", 400, 30),
    ("Réfrigérateur (inv.)", 20, -10),
]

print("Rendements de Carnot :")
for nom, Tc, Tf in machines:
    eta = rendement_carnot(Tc, Tf)
    print(f" {nom:<25} Tc={Tc}°C, Tf={Tf}°C → eta_max = {eta*100:.1f}%")

# 1re loi : gaz chauffé à volume constant
n = 1.0
Cv = 3/2 * R_gaz # gaz monoatomique : Cv = (3/2)R
delta_T = 100 # K
delta_U = n * Cv * delta_T
Q_isochore = delta_U # W = 0 à volume constant
print(f"\nGaz chauffé à V constant (delta_T=100K) : delta_U = {delta_U:.2f} J")
```

---

[PIÈGE]

La 2e loi ne dit pas que l'entropie augmente PARTOUT, mais dans un

SYSTÈME ISOLÉ. Un réfrigérateur peut diminuer l'entropie d'un espace, mais il augmente l'entropie totale de l'univers (il chauffe la pièce).

---

---

## BRANCHE 3 – ÉLECTROMAGNÉTISME

---

---

---

### 3.1 ÉLECTROSTATIQUE – charges au repos

---

#### [DÉFINITION]

LOI DE COULOMB :

$$F = k * |q1 * q2| / r^2$$

$$k = 1/(4*\pi*\epsilon_0) = 8.99e9 \text{ N}\cdot\text{m}^2/\text{C}^2$$

$$\epsilon_0 = 8.854e-12 \text{ F/m} \quad (\text{permittivité du vide})$$

Attractive si charges opposées, répulsive si charges identiques.

CHAMP ÉLECTRIQUE  $E$  (créé par une charge  $Q$  en un point  $r$ ) :

$$E = k * Q / r^2 \quad [\text{V/m} = \text{N/C}]$$

$$\text{Force sur une charge test } q : F = q * E$$

POTENTIEL ÉLECTRIQUE  $V$  :

$$V = k * Q / r \quad [\text{Volt, V}]$$

$$\text{Relation } E \text{ et } V : E = -dV/dx$$

ÉNERGIE POTENTIELLE ÉLECTRIQUE :

$$U = k * q1 * q2 / r \quad [\text{J}]$$

PRINCIPE DE SUPERPOSITION :

Le champ total est la somme vectorielle des champs individuels.

#### [CODE PYTHON]

---

```
k_coulomb = 8.99e9    # N·m²/C²
e_charge  = 1.602e-19 # C (charge élémentaire)

def force_coulomb(q1, q2, r):
    return k_coulomb * abs(q1 * q2) / r**2

def champ_electrique(Q, r):
    return k_coulomb * Q / r**2

def potentiel(Q, r):
    return k_coulomb * Q / r

def energie_potentielle_elec(q1, q2, r):
    return k_coulomb * q1 * q2 / r

# Proton-électron dans l'atome d'hydrogène (rayon de Bohr)
```

```

r_bohr = 5.29e-11 # m
q_p = +e_charge
q_e = -e_charge

F = force_coulomb(q_p, q_e, r_bohr)
U = energie_potentielle_elec(q_p, q_e, r_bohr)
V_proton = potentiel(q_p, r_bohr)

print(f"Atome H (rayon de Bohr = {r_bohr:.2e} m) :")
print(f" Force p-e      : {F:.3e} N")
print(f" Énergie pot.    : {U:.3e} J = {U/e_charge:.2f} eV")
print(f" Potentiel       : {V_proton:.2f} V")

# Champ d'un dipôle (deux charges ±q séparées de d)
q = 1e-9 # nC
d = 0.01 # 1 cm
E_plus = champ_electrique(+q, d/2)
E_minus = champ_electrique(-q, d/2)
print(f"\nDipôle : E au milieu (contributions) = {E_plus:.2f} V/m chacun")

```

---

## 3.2 CIRCUITS ÉLECTRIQUES

---

### [DÉFINITION]

LOI D'OHM :

$$U = R * I$$

U [Volt], R [Ohm Ω], I [Ampère A]

PUISSANCE :

$$P = U * I = R * I^2 = U^2 / R \quad [\text{Watt W}]$$

LOIS DE KIRCHHOFF :

Loi des nœuds :  $\Sigma(I_{\text{entrant}}) = \Sigma(I_{\text{sortant}})$

Loi des mailles :  $\Sigma(U) = 0$  sur toute maille fermée

RÉSISTANCES :

En série :  $R_{\text{total}} = R_1 + R_2 + \dots + R_n$

En parallèle:  $1/R_{\text{total}} = 1/R_1 + 1/R_2 + \dots + 1/R_n$

CONDENSATEUR :

$$Q = C * U \quad C \text{ en Farad [F]}$$

Energie stockée :  $E = (1/2) * C * U^2$

Charge/décharge :  $U(t) = U_0 * (1 - \exp(-t/(R*C)))$  [charge]

$U(t) = U_0 * \exp(-t/(R*C))$  [décharge]

INDUCTANCE :

$$U = L * dI/dt \quad L \text{ en Henry [H]}$$

Énergie stockée :  $E = (1/2) * L * I^2$

[CODE PYTHON]

---

```

import math

def resistance_serie(*R_list):
    return sum(R_list)

def resistance_parallele(*R_list):
    return 1 / sum(1/R for R in R_list)

def circuit_RC_charge(U0, R, C, t):
    """Tension aux bornes du condensateur pendant la charge"""
    tau = R * C
    return U0 * (1 - math.exp(-t / tau))

def circuit_RC_decharge(U0, R, C, t):
    tau = R * C
    return U0 * math.exp(-t / tau)

# Résistances
R1, R2, R3 = 10, 20, 30    # Ω
print(f"R1+R2+R3 en série    : {resistance_serie(R1,R2,R3):.2f} Ω")
print(f"R1||R2||R3 parallèle : {resistance_parallele(R1,R2,R3):.2f} Ω")

# Loi d'Ohm
U, R = 12, 100
I = U / R
P = U * I
print(f"\nU={U}V, R={R}Ω → I={I*1000:.1f} mA, P={P*1000:.1f} mW")

# Circuit RC : décharge
U0 = 5.0    # V
R = 1000    # Ω (1 kΩ)
C = 100e-6 # F (100 μF)
tau = R * C # constante de temps
print(f"\nDécharge RC : tau = {tau*1000:.1f} ms")
print(f"{'t/tau':>6} | {'U (V)':>8} | {'% U0':>6}")
for ratio in [0, 0.5, 1, 2, 3, 5]:
    t = ratio * tau
    U_t = circuit_RC_decharge(U0, R, C, t)
    print(f"{'ratio':>6.1f} | {'U_t':>8.4f} | {'U_t/U0*100:>5.1f}%")

```

---



---

### 3.3 MAGNÉTISME ET ÉQUATIONS DE MAXWELL

---

#### [DÉFINITION]

FORCE DE LORENTZ (sur une charge  $q$  dans  $E$  et  $B$ ) :

$$\mathbf{F} = q(\mathbf{E} + \mathbf{v} \times \mathbf{B})$$

FORCE DE LAPLACE (sur un fil de courant dans  $B$ ) :

$$\mathbf{F} = I * \mathbf{L} \times \mathbf{B} \quad (\mathbf{L} : \text{vecteur longueur dans la direction du courant})$$

LOI DE BIOT-SAVART (champ B créé par un fil parcouru par I) :

$$dB = (\mu_0/4\pi) * I * dL \times r / r^3$$

Pour un fil infini :  $B = \mu_0 * I / (2 * \pi * r)$

$$\mu_0 = 4 * \pi * 1e-7 \text{ T}\cdot\text{m/A} \quad (\text{perméabilité du vide})$$

ÉQUATIONS DE MAXWELL (forme intégrale) :

(1) Gauss électrique :  $\text{flux}(E) = Q_{\text{enc}} / \epsilon_0$

(2) Gauss magnétique :  $\text{flux}(B) = 0$  (pas de monopôle magnétique)

(3) Faraday :  $\text{circulation}(E) = -d(\text{flux}_B)/dt$

(4) Ampère-Maxwell :  $\text{circulation}(B) = \mu_0 * (I + \epsilon_0 * d(\text{flux}_E)/dt)$

Ces 4 équations gouvernent tout l'électromagnétisme classique.

Leur unification par Maxwell a prédit l'existence des ondes EM

et montré que la lumière EST une onde électromagnétique.

Vitesse de la lumière (dans le vide) :

$$c = 1 / \sqrt{\mu_0 * \epsilon_0} = 3e8 \text{ m/s}$$

[CODE PYTHON]

---

```
mu0 = 4 * math.pi * 1e-7    # T·m/A
epsilon0 = 8.854e-12         # F/m
c_lumiere = 1 / math.sqrt(mu0 * epsilon0)

print(f"c calculé depuis mu0 et epsilon0 : {c_lumiere:.4e} m/s")
print(f"c connu : 2.9979e8 m/s → {'OK' if abs(c_lumiere-3e8)/3e8 < 0.01 else 'écart'}")

def champ_B_fil_infini(I, r):
    """Champ magnétique à distance r d'un fil infini parcouru par I"""
    return mu0 * I / (2 * math.pi * r)

def force_lorentz(q, v, B):
    """Norme de la force magnétique F = qvB (v ⊥ B)"""
    return abs(q) * v * B

# Fil parcouru par 10A
I = 10    # A
for r_cm in [1, 5, 10, 50]:
    r = r_cm / 100
    B = champ_B_fil_infini(I, r)
    print(f"  r = {r_cm:3d} cm → B = {B:.4e} T")

# Électron dans un champ magnétique (cyclotron)
m_e = 9.109e-31 # kg
v_e = 1e7       # m/s (1% de c)
B = 0.1         # T
F_mag = force_lorentz(q_e, v_e, B)
r_cyclotron = m_e * v_e / (abs(q_e) * B)
print(f"\nÉlectron v={v_e:.1e}m/s dans B={B}T :")
print(f"  F = {F_mag:.3e} N | r_cyclotron = {r_cyclotron*100:.4f} cm")
```

---

---

## 4.1 OPTIQUE GÉOMÉTRIQUE

---

### [DÉFINITION]

LOI DE SNELL-DESCARTES (réfraction) :

$$n_1 * \sin(\theta_1) = n_2 * \sin(\theta_2)$$

$n$  : indice de réfraction (sans unité,  $n \geq 1$ )

$n_{\text{vide}} = 1$ ,  $n_{\text{eau}} \approx 1.33$ ,  $n_{\text{verre}} \approx 1.5$

RÉFLEXION :

Angle d'incidence = angle de réflexion (par rapport à la normale)

ANGLE CRITIQUE (réflexion totale interne) :

$$\sin(\theta_c) = n_2 / n_1 \quad (\text{avec } n_1 > n_2)$$

Si  $\theta > \theta_c$  : réflexion totale (principe des fibres optiques)

LENTILLES MINCES (formule de conjugaison) :

$$1/OA' - 1/OA = 1/f' \quad (\text{convention algébrique})$$

$$\text{ou : } 1/v - 1/u = 1/f$$

$f'$  : distance focale [m],  $f' > 0$  (convergente),  $f' < 0$  (divergente)

Vergence :  $V = 1/f'$  [dioptries,  $\delta$ ]

GRANDISSEMENT :

$$\gamma = OA'/OA = \text{taille image} / \text{taille objet}$$

### [CODE PYTHON]

---

```
def snell_descartes(n1, theta1_deg, n2):
    """Retourne theta2 (réfraction) ou None si réflexion totale"""
    theta1 = math.radians(theta1_deg)
    sin_theta2 = n1 * math.sin(theta1) / n2
    if abs(sin_theta2) > 1:
        return None # réflexion totale interne
    return math.degrees(math.asin(sin_theta2))

def angle_critique(n1, n2):
    if n1 <= n2:
        return None # pas de réflexion totale
    return math.degrees(math.asin(n2 / n1))

def lentille_conjugaison(OA, f_prime):
    """Retourne OA' (position image)"""
    # 1/OA' = 1/f' + 1/OA
    return 1 / (1/f_prime + 1/OA)
```

```

def grandissement(OA, OA_prime):
    return OA_prime / OA

# Réfraction air→verre
n_air, n_verre = 1.0, 1.5
print("Réfraction air→verre :")
for theta1 in [0, 20, 40, 60, 80]:
    theta2 = snell_descartes(n_air, theta1, n_verre)
    print(f"  theta1={theta1:2d}° → theta2={theta2:.2f}°" if theta2 else f"  theta1={theta1}°
→ RÉFLEXION TOTALE")

# Angle critique verre→air
theta_c = angle_critique(n_verre, n_air)
print(f"\nAngle critique verre→air : {theta_c:.2f}° (fibres optiques)")

# Lentille convergente f'=10cm, objet à 30cm
OA = -30e-2 # m (objet à gauche, convention : OA négatif)
f_prime = 10e-2 # m
OA_prime = lentille_conjugaison(OA, f_prime)
gamma = grandissement(OA, OA_prime)
print(f"\nLentille f'={f_prime*100}cm, OA={OA*100}cm :")
print(f"  OA' = {OA_prime*100:.2f} cm (image {'réelle' if OA_prime > 0 else 'virtuelle'})")
print(f"  gamma = {gamma:.3f} (image {'droite' if gamma > 0 else 'renversée'}, taille
{'réduite' if abs(gamma)<1 else 'agrandie'})")

```

---



---

## 4.2 OPTIQUE ONDULATOIRE – diffraction et interférences

---

### [DÉFINITION]

INTERFÉRENCES (fentes de Young) :

Franges brillantes :  $\Delta = k \cdot \lambda$  (k entier)

Franges sombres :  $\Delta = (k + 1/2) \cdot \lambda$

Inter-franges :  $i = \lambda \cdot D / a$

$\lambda$  : longueur d'onde [m], D : distance écran, a : écart entre fentes

DIFFRACTION (fente simple de largeur b) :

Premier minimum :  $\sin(\theta) = \lambda / b$

Plus b est petit par rapport à  $\lambda$ , plus la diffraction est grande.

LONGUEURS D'ONDE VISIBLES :

Violet : 380–450 nm    Bleu : 450–495 nm    Vert : 495–570 nm

Jaune : 570–590 nm    Orange : 590–625 nm    Rouge : 625–750 nm

### [CODE PYTHON]

---

```

def interfrange_young(lam, D, a):
    """Distance entre deux franges brillantes consécutives"""
    return lam * D / a

def angle_premier_minimum(lam, b):

```



```

"""Angle du premier minimum de diffraction (fente simple)"""
return math.degrees(math.asin(lam / b))

# Fentes de Young : laser vert lambda=532nm
lam = 532e-9    # m
D    = 1.0      # m
a    = 0.5e-3    # m (0.5 mm entre fentes)
i = interfrange_young(lam, D, a)
print(f"Young lambda=532nm, D=1m, a=0.5mm : i = {i*1000:.3f} mm")

# Positions des franges brillantes
print("Franges brillantes :")
for k in range(-3, 4):
    pos = k * i
    print(f"  k={k:+d} : y = {pos*1000:.3f} mm")

# Diffraction : fente b=0.1mm, laser rouge 633nm
b = 0.1e-3    # m
lam_r = 633e-9
theta_min = angle_premier_minimum(lam_r, b)
print(f"\nDiffraction : fente {b*1e3}mm, lambda={lam_r*1e9:.0f}nm")
print(f"  1er minimum à theta = {theta_min:.3f}°")

```

---



---

## BRANCHE 5 – PHYSIQUE DES ONDES

---



---

### [DÉFINITION]

UNE ONDE est une perturbation qui se propage en transportant de l'énergie SANS transporter de matière.

### PARAMÈTRES D'UNE ONDE SINUSOÏDALE :

$y(x, t) = A \cdot \sin(kx - \omega t + \phi)$   
 A : amplitude [m ou unité physique]  
 k : nombre d'onde =  $2\pi/\lambda$  [rad/m]  
 $\omega$  : pulsation =  $2\pi f$  [rad/s]  
 $\lambda$  : longueur d'onde [m]  
 f : fréquence [Hz = 1/s]  
 T : période =  $1/f$  [s]  
 v : vitesse de phase =  $\lambda f = \omega/k$  [m/s]

### ÉQUATION D'ONDE :

$$d^2y/dt^2 = v^2 \cdot d^2y/dx^2$$

### EFFET DOPPLER :

Source en mouvement vs (vers l'observateur) :  
 $f_{\text{obs}} = f_{\text{source}} \cdot v_{\text{son}} / (v_{\text{son}} - v_s)$  (source qui approche)  
 $f_{\text{obs}} = f_{\text{source}} \cdot v_{\text{son}} / (v_{\text{son}} + v_s)$  (source qui s'éloigne)  
 $v_{\text{son}} \approx 340 \text{ m/s}$  dans l'air à 20°C

INTENSITÉ SONORE :

$I = P / (4\pi r^2)$  [W/m<sup>2</sup>] (onde sphérique)

Niveau sonore :  $L = 10 * \log_{10}(I / I_0)$  [dB]

$I_0 = 1e-12$  W/m<sup>2</sup> (seuil audition)

[CODE PYTHON]

---

```
import math

v_son = 340    # m/s à 20°C

def onde(A, lam, f, x, t, phi=0):
    """Déplacement d'une onde sinusoïdale"""
    k = 2 * math.pi / lam
    omega = 2 * math.pi * f
    return A * math.sin(k*x - omega*t + phi)

def doppler_approche(f_source, vs):
    return f_source * v_son / (v_son - vs)

def doppler_eloigne(f_source, vs):
    return f_source * v_son / (v_son + vs)

def niveau_sonore_dB(I):
    I0 = 1e-12
    return 10 * math.log10(I / I0)

def intensite_depuis_dB(dB):
    return 1e-12 * 10**(dB / 10)

# Onde sonore : A=0.001m, lambda=1m, f=340Hz
A, lam, f = 0.001, 1.0, 340
print("Onde y(x,t) = A*sin(kx - omega*t) :")
for x in [0, 0.25, 0.5, 0.75, 1.0]:
    y = onde(A, lam, f, x, t=0)
    print(f"  y({x:.2f}, 0) = {y*1000:.4f} mm")

# Effet Doppler – ambulance à 60 km/h
vs = 60 / 3.6    # m/s
f_sirene = 700  # Hz
print(f"\nAmbulance f={f_sirene}Hz, v={vs:.1f}m/s :")
print(f"  Approche : f_obs = {doppler_approche(f_sirene, vs):.1f} Hz")
print(f"  Éloigne  : f_obs = {doppler_eloigne(f_sirene, vs):.1f} Hz")

# Niveaux sonores
exemples = [("Seuil audition", 0), ("Chuchotement", 30), ("Conversation", 60),
            ("Concert rock", 110), ("Seuil douleur", 130)]
print("\nNiveaux sonores :")
for nom, dB in exemples:
    I = intensite_depuis_dB(dB)
    print(f"  {nom:<20} {dB:>4} dB → I = {I:.2e} W/m²")
```

---

## [PIÈGE]

La vitesse de phase  $v = \lambda \cdot f$  est la vitesse à laquelle une crête se propage, PAS la vitesse des particules du milieu. Les particules oscillent autour de leur position d'équilibre : elles ne "voyagent" pas avec l'onde.

## BRANCHE 6 – MÉCANIQUE QUANTIQUE

## [THÉORIE]

À l'échelle atomique, la physique classique échoue. La mécanique quantique décrit les particules comme des FONCTIONS D'ONDE : on ne peut pas connaître simultanément la position et la vitesse d'une particule (principe d'incertitude). Tout est probabiliste.

## [DÉFINITION]

CONSTANTES FONDAMENTALES :

$$h = 6.626 \times 10^{-34} \text{ J}\cdot\text{s} \quad (\text{constante de Planck})$$

$$\hbar = h / (2\pi) = 1.055 \times 10^{-34} \text{ J}\cdot\text{s}$$

DUALITÉ ONDE-CORPUSCULE :

$$\text{Photon} : E = h \cdot f = \hbar \cdot \omega \quad | \quad p = h / \lambda = \hbar \cdot k$$

$$\text{Matière} : \lambda_{\text{de Broglie}} = h / (m \cdot v)$$

PRINCIPE D'INCERTITUDE DE HEISENBERG :

$$\Delta x \cdot \Delta p \geq \hbar / 2$$

$$\Delta E \cdot \Delta t \geq \hbar / 2$$

On ne peut jamais connaître position ET quantité de mouvement avec une précision arbitraire simultanément.

ÉQUATION DE SCHRÖDINGER (stationnaire) :

$$H\psi = E\psi$$

$$H = -\hbar^2 / (2m) \cdot d^2/dx^2 + V(x) \quad (\text{Hamiltonien})$$

$\psi$  : fonction d'onde (la solution)

$|\psi|^2$  : densité de probabilité de présence

QUANTIFICATION DE L'ÉNERGIE (puits infini de largeur L) :

$$E_n = n^2 \cdot \pi^2 \cdot \hbar^2 / (2m \cdot L^2) \quad n = 1, 2, 3, \dots$$

Les niveaux d'énergie sont DISCRETS (quantifiés).

NIVEAUX D'ÉNERGIE DE L'HYDROGÈNE :

$$E_n = -13.6 / n^2 \quad [\text{eV}] \quad n = 1, 2, 3, \dots$$

$$E_1 = -13.6 \text{ eV} \quad (\text{état fondamental})$$

## [CODE PYTHON]

```
h_planck = 6.626e-34 # J·s
hbar      = h_planck / (2 * math.pi)
```

```

m_e = 9.109e-31 # kg
e_charge = 1.602e-19 # C

def de_broglie(m, v):
    """Longueur d'onde de De Broglie"""
    return h_planck / (m * v)

def energie_puits_infini(n, m, L):
    """Énergie du niveau n dans un puits infini de largeur L"""
    return n**2 * math.pi**2 * hbar**2 / (2 * m * L**2)

def energie_hydrogene(n):
    """Énergie du niveau n de l'atome d'hydrogène [eV]"""
    return -13.6 / n**2

def longueur_onda_photon_H(n1, n2):
    """Lambda du photon émis pour la transition n2 → n1 (n2 > n1)"""
    delta_E = (energie_hydrogene(n2) - energie_hydrogene(n1)) * e_charge # en J
    f = abs(delta_E) / h_planck
    return 3e8 / f # lambda = c/f

# Longueur d'onde de De Broglie
print("Longueurs d'onde de De Broglie :")
objets = [("Électron 1keV", m_e, math.sqrt(2*1000*e_charge/m_e)),
          ("Balle de tennis 60m/s", 0.057, 60),
          ("Proton 1MeV", 1.673e-27, math.sqrt(2*1e6*e_charge/1.673e-27))]
for nom, m, v in objets:
    lam = de_broglie(m, v)
    print(f" {nom:<30} : lambda = {lam:.3e} m")

# Niveaux d'énergie de l'hydrogène
print("\nNiveaux d'énergie de l'hydrogène :")
for n in range(1, 7):
    E = energie_hydrogene(n)
    print(f" n={n} : E = {E:.4f} eV")

# Transitions (série de Balmer : vers n=2)
print("\nSérie de Balmer (transitions → n=2) :")
for n2 in range(3, 7):
    lam = longueur_onda_photon_H(2, n2)
    region = "visible" if 380e-9 <= lam <= 750e-9 else "UV/IR"
    print(f" n={n2}→2 : lambda = {lam*1e9:.1f} nm ({region})")

# Principe d'incertitude
print("\nPrincipe d'incertitude :")
delta_x = 1e-10 # position connue à 0.1nm
delta_p_min = hbar / (2 * delta_x)
delta_v_min = delta_p_min / m_e
print(f" Si delta_x = {delta_x:.1e} m (Bohr)")
print(f" → delta_p >= {delta_p_min:.3e} kg·m/s")
print(f" → delta_v >= {delta_v_min:.3e} m/s")

```

---

[PIÈGE]

La fonction d'onde  $\psi$  n'est PAS une onde physique réelle.  
C'est une amplitude de probabilité :  $|\psi|^2$  est la probabilité  
de trouver la particule à cet endroit. La particule n'est pas  
"étalée dans l'espace" – c'est notre CONNAISSANCE qui est incertaine.

---

---

||

BRANCHE 7 – RELATIVITÉ

||

---

---

||

[THÉORIE]

La relativité restreinte (Einstein, 1905) s'applique à grande vitesse.  
La relativité générale (Einstein, 1915) intègre la gravitation comme  
courbure de l'espace-temps.

[DÉFINITION – RELATIVITÉ RESTREINTE]

POSTULATS :

1. Les lois de la physique sont les mêmes dans tout référentiel inertiel.
2. La vitesse de la lumière  $c = 3e8$  m/s est la même pour tous  
les observateurs.

FACTEUR DE LORENTZ :

$$\gamma = 1 / \sqrt{1 - v^2/c^2}$$

$\gamma \geq 1$ , et  $\gamma \rightarrow \infty$  quand  $v \rightarrow c$

DILATATION DU TEMPS :

$$\Delta t = \gamma * \Delta t_0$$

(une horloge en mouvement bat plus lentement)

CONTRACTION DES LONGUEURS :

$$L = L_0 / \gamma$$

(un objet en mouvement est plus court dans la direction du mouvement)

ÉNERGIE TOTALE :

$$E = \gamma * m * c^2$$

Énergie au repos :  $E_0 = m * c^2$   
Énergie cinétique :  $E_c = (\gamma - 1) * m * c^2$

ADDITION DES VITESSES :

$$u' = (u + v) / (1 + u*v/c^2)$$

(jamais supérieure à  $c$ )

[CODE PYTHON]

---

```
c = 3e8    # m/s

def gamma_lorentz(v):
    """Facteur de Lorentz"""
    if abs(v) >= c:
        raise ValueError("v doit être < c")
```

```

return 1 / math.sqrt(1 - (v/c)**2)

def dilatation_temps(t0, v):
    return gamma_lorentz(v) * t0

def contraction_longueur(L0, v):
    return L0 / gamma_lorentz(v)

def energie_totale(m, v):
    return gamma_lorentz(v) * m * c**2

def addition_vitesses_rel(u, v):
    return (u + v) / (1 + u*v/c**2)

m_e = 9.109e-31 # kg
print("Effets relativistes en fonction de v/c :")
print(f"{'v/c':>6} | {'gamma':>8} | {'t/t0':>8} | {'L/L0':>8}")
print("-" * 40)
for ratio in [0.1, 0.5, 0.8, 0.9, 0.99, 0.999]:
    v = ratio * c
    g = gamma_lorentz(v)
    print(f"{'ratio':>6.3f} | {'g':>8.4f} | {'g':>8.4f} | {'1/g':>8.4f}")

# E = mc² : énergie au repos d'un électron
E0_elec = m_e * c**2
print(f"\nE0 électron = {E0_elec:.4e} J = {E0_elec/e_charge/1e3:.4f} keV")

# Addition des vitesses : deux particules à 0.9c en sens opposés
u = 0.9 * c
v = -0.9 * c # l'autre dans la direction opposée
v_rel = addition_vitesses_rel(u, -v) # vitesse relative
print(f"\nAddition relativiste 0.9c + 0.9c = {v_rel/c:.4f} c (jamais ≥ 1 !)")

```

#### [PIÈGE]

$E = mc^2$  s'applique à l'énergie AU REPOS. L'énergie totale est  $E = \gamma mc^2$ . À faible vitesse,  $\gamma \approx 1 + v^2/(2c^2)$ , ce qui redonne bien  $(1/2)mv^2$  comme terme cinétique dominant.

#### [THÉORIE]

La physique statistique fait le pont entre les lois microscopiques (chaque atome obéit à Newton ou à Schrödinger) et les grandeurs macroscopiques (température, pression, entropie). C'est la loi des grands nombres appliquée à la physique.

#### [DÉFINITION]

DISTRIBUTION DE MAXWELL-BOLTZMANN (vitesses des molécules d'un gaz) :

$$f(v) = 4\pi * (m/(2\pi*k_B*T))^{(3/2)} * v^2 * \exp(-m*v^2/(2*k_B*T))$$

$$k_B = 1.381e-23 \text{ J/K} \quad (\text{constante de Boltzmann})$$

Vitesses caractéristiques :

$$\text{Vitesse la plus probable : } v_p = \sqrt{2*k_B*T/m}$$

$$\text{Vitesse moyenne : } v_{\text{moy}} = \sqrt{8*k_B*T/(\pi*m)}$$

$$\text{Vitesse quadratique moyenne : } v_{\text{rms}} = \sqrt{3*k_B*T/m}$$

ENTROPIE DE BOLTZMANN :

$$S = k_B * \ln(\Omega)$$

$\Omega$  : nombre de micro-états compatibles avec l'état macroscopique

FACTEUR DE BOLTZMANN (probabilité d'un état d'énergie E à température T) :

$$P(E) \text{ proportionnel à } \exp(-E/(k_B*T))$$

[CODE PYTHON]

---

```
kB = 1.381e-23 # J/K
```

```
def maxwell_boltzmann(v, m, T):
```

```
    """Distribution des vitesses de Maxwell-Boltzmann"""
```

```
    A = 4 * math.pi * (m / (2 * math.pi * kB * T))**(3/2)
```

```
    return A * v**2 * math.exp(-m * v**2 / (2 * kB * T))
```

```
def vitesse_probable(m, T):
```

```
    return math.sqrt(2 * kB * T / m)
```

```
def vitesse_rms(m, T):
```

```
    return math.sqrt(3 * kB * T / m)
```

```
def vitesse_moyenne(m, T):
```

```
    return math.sqrt(8 * kB * T / (math.pi * m))
```

```
# Molécule de diazote N2 (m ≈ 28 g/mol)
```

```
m_N2 = 28e-3 / 6.022e23 # masse d'une molécule (kg)
```

```
T = 293.15 # 20°C
```

```
vp = vitesse_probable(m_N2, T)
```

```
vmoy = vitesse_moyenne(m_N2, T)
```

```
vrms = vitesse_rms(m_N2, T)
```

```
print(f"Vitesses N2 à 20°C :")
```

```
print(f"  vp   (plus probable)      = {vp:.1f} m/s")
```

```
print(f"  v_moy (moyenne)              = {vmoy:.1f} m/s")
```

```
print(f"  v_rms (quadratique moy.)     = {vrms:.1f} m/s")
```

```
# Distribution : profil de f(v)
```

```
print("\nDistribution f(v) pour N2 à 293K :")
```

```
print(f"{'v (m/s)':>10} | {'f(v)':>12}")
```

```
for v in range(0, 1600, 100):
```

```
    fv = maxwell_boltzmann(v, m_N2, T)
```

```
    bar = int(fv * 1e3 * 30)
```

```
print(f"{v:>10} | {fv:.4e} - {'█'*min(bar, 40)}")

# Facteur de Boltzmann – population des niveaux
print("\nFacteur de Boltzmann exp(-E/kBT) :")
for E_eV in [0.001, 0.01, 0.1, 0.5, 1.0]:
    E = E_eV * e_charge
    facteur = math.exp(-E / (kB * T))
    print(f" E={E_eV} eV : facteur = {facteur:.4e}")
```

---

BRANCHE 9 – PHYSIQUE NUCLÉAIRE

---

# [DÉFINITION]

## COMPOSITION DU NOYAU :

Nucléons = protons (Z) + neutrons (N)  
 Nombre de masse A = Z + N  
 Notation :  $A_Z X$  (ex :  $^{235}_{92} \text{U}$  = Uranium-235)

## DÉFAUT DE MASSE ET ÉNERGIE DE LIAISON :

$\Delta m = Z \cdot m_p + N \cdot m_n - M_{\text{noyau}}$  [kg ou u]  
 $E_{\text{liaison}} = \Delta m \cdot c^2$  [MeV]  
 $u = 1.661 \times 10^{-27} \text{ kg}$  (unité de masse atomique),  $1 u = 931.5 \text{ MeV}/c^2$

## RADIOACTIVITÉ – LOI DE DÉCROISSANCE :

$N(t) = N_0 \cdot \exp(-\lambda_{\text{rad}} \cdot t)$   
 $\lambda_{\text{rad}} = \ln(2) / t_{1/2}$  (constante de désintégration)  
 $t_{1/2}$  : demi-vie (temps pour que N diminue de moitié)  
 Activité  $A(t) = \lambda_{\text{rad}} \cdot N(t)$  [Becquerel, Bq = désintégration/s]

## TYPES DE RADIOACTIVITÉ :

Alpha ( $\alpha$ ) : émission d'un noyau He-4 ( $2p + 2n$ )  
 Bêta- ( $e^-$ ) : un neutron  $\rightarrow$  proton + électron + antineutrino  
 Bêta+ ( $e^+$ ) : un proton  $\rightarrow$  neutron + positron + neutrino  
 Gamma (photon) : réorganisation du noyau, pas de changement A/Z

## FISSION vs FUSION :

Fission : un noyau lourd se casse en deux  $\rightarrow$  libère énergie (centrale)  
 Fusion : deux noyaux légers fusionnent  $\rightarrow$  libère encore PLUS d'énergie (soleil)

# [CODE PYTHON]

```
def decroissance_radioactive(N0, t_demi, t):
    """Nombre de noyaux restants après temps t"""
    lambda_rad = math.log(2) / t_demi
    return N0 * math.exp(-lambda_rad * t)

def activite(N0, t_demi, t):
    lambda_rad = math.log(2) / t_demi
```



```

N = decroissance_radioactive(N0, t_demi, t)
return lambda_rad * N

def energie_liaison(Z, N, M_noyau_u):
    """Énergie de liaison en MeV"""
    m_p = 1.007276    # u
    m_n = 1.008665    # u
    delta_m = Z * m_p + N * m_n - M_noyau_u    # en u
    return delta_m * 931.5    # MeV

# Carbone-14 : t_1/2 = 5730 ans
t_demi_C14 = 5730    # ans
N0 = 1e12            # noyaux initiaux

print("Décroissance Carbone-14 (datation) :")
print(f"{'Temps (ans)':>12} | {'N restants':>12} | {'Fraction':>8}")
for t in [0, 1000, 5730, 10000, 20000, 50000]:
    N = decroissance_radioactive(N0, t_demi_C14, t)
    print(f"{'t':>12} | {'N:.3e'} | {'N/N0*100:>7.3f}%")

# Énergie de liaison de l'Hélium-4
# He-4 : Z=2, N=2, M=4.002602 u
E_He4 = energie_liaison(2, 2, 4.002602)
E_par_nucleon = E_He4 / 4
print(f"\nHélium-4 : E_liaison = {E_He4:.4f} MeV ({E_par_nucleon:.4f} MeV/nucléon)")

# Énergie libérée par fission d'U-235
# Environ 200 MeV par fission
E_fission_MeV = 200    # MeV
E_fission_J = E_fission_MeV * 1e6 * e_charge
n_avogadro = 6.022e23
M_U235 = 235e-3    # kg/mol
n_noyaux_par_kg = n_avogadro / M_U235 * 1000    # noyaux/kg
E_par_kg = n_noyaux_par_kg * E_fission_J
print(f"\nFission U-235 : énergie par kg = {E_par_kg:.2e} J")
print(f"    ≡ {E_par_kg / 4.184e9:.2e} tonnes de TNT équivalent")

```

---



---

||

||

---



---

||

#### [DÉFINITION]

PRESSION :

$$P = F / A \quad [\text{Pascal, Pa} = \text{N/m}^2]$$

Pression atmosphérique :  $P_{\text{atm}} \approx 101325 \text{ Pa} = 1 \text{ atm}$

PRESSION HYDROSTATIQUE :

$$P(h) = P_0 + \rho * g * h$$

$\rho$  : masse volumique [ $\text{kg/m}^3$ ] (eau : 1000, air : 1.2  $\text{kg/m}^3$ )

POUSSÉE D'ARCHIMÈDE :

$$F_{\text{Archimède}} = \rho_{\text{fluide}} * g * V_{\text{immergé}}$$

Un objet flotte si sa densité < densité du fluide.

ÉQUATION DE CONTINUITÉ (fluide incompressible) :

$$A_1 * v_1 = A_2 * v_2$$

(le débit volumique  $Q = A*v$  est constant)

ÉQUATION DE BERNOULLI (fluide parfait, stationnaire) :

$$P + (1/2)*\rho*v^2 + \rho*g*h = \text{constante}$$

Pression statique + pression dynamique + pression hydrostatique = cste

VISCOSITÉ (fluide réel) :

$$\text{Force de cisaillement : } F = \eta * A * dv/dy$$

$\eta$  : viscosité dynamique [Pa·s]

eau :  $\eta \approx 1e-3$  Pa·s, huile :  $\eta \approx 0.1$  Pa·s, air :  $\eta \approx 1.8e-5$  Pa·s

NOMBRE DE REYNOLDS :

$$Re = \rho * v * L / \eta$$

$Re < 2000$  : écoulement laminaire

$Re > 4000$  : écoulement turbulent

[DÉMONSTRATION] Vitesse de sortie d'une cuve (Torricelli)

Cuve d'eau, surface libre à hauteur  $h$ , orifice à la base.

Appliquer Bernoulli entre la surface ( $v \approx 0$ ,  $P = P_{\text{atm}}$ ,  $y = h$ )

et l'orifice ( $P = P_{\text{atm}}$ ,  $y = 0$ ) :

$$P_{\text{atm}} + 0 + \rho*g*h = P_{\text{atm}} + (1/2)*\rho*v^2 + 0$$

$$\rho*g*h = (1/2)*\rho*v^2$$

$$v = \sqrt{2*g*h} \quad (\text{formule de Torricelli}) \quad \square$$

[CODE PYTHON]

---

```
rho_eau = 1000    # kg/m³
```

```
rho_air = 1.225   # kg/m³
```

```
eta_eau = 1e-3    # Pa·s
```

```
def pression_hydrostatique(P0, rho, h):
```

```
    return P0 + rho * g * h
```

```
def pousse_archimede(rho_fluide, V_immerge):
```

```
    return rho_fluide * g * V_immerge
```

```
def vitesse_torricelli(h):
```

```
    return math.sqrt(2 * g * h)
```

```
def debit_volumique(A, v):
```

```
    return A * v
```

```
def bernoulli(P1, v1, h1, rho, P2=None, v2=None, h2=None):
```

```
    """
```

```
    Retourne la grandeur manquante (None parmi P2, v2, h2).
```

```
     $P + (1/2)*\rho*v^2 + \rho*g*h = \text{constante}$ 
```

```

"""
C = P1 + 0.5*rho*v1**2 + rho*g*h1
if P2 is None:
    return C - 0.5*rho*v2**2 - rho*g*h2
elif v2 is None:
    val = (C - P2 - rho*g*h2) * 2 / rho
    return math.sqrt(max(0, val))
elif h2 is None:
    return (C - P2 - 0.5*rho*v2**2) / (rho * g)

def reynolds(rho, v, L, eta):
    return rho * v * L / eta

P_atm = 101325    # Pa

# Pression hydrostatique sous l'eau
print("Pression sous l'eau (P0 = 1 atm) :")
for depth in [0, 10, 100, 1000, 10000]:
    P = pression_hydrostatique(P_atm, rho_eau, depth)
    print(f"  h = {depth:>5} m → P = {P:.3e} Pa ({P/P_atm:.1f} atm)")

# Archimède : flotte ou coule ?
print("\nFlottabilité :")
objets_rho = [("Bois", 700), ("Glace", 917), ("Béton", 2400), ("Fer", 7874)]
for nom, rho in objets_rho:
    flotte = rho < rho_eau
    print(f"  {nom:<10} rho={rho} kg/m³ → {'FLOTTE' if flotte else 'COULE'}")

# Torricelli
h = 2.0    # m
v_sortie = vitesse_torricelli(h)
print(f"\nTorricelli : cuve h={h}m → v_sortie = {v_sortie:.3f} m/s")

# Bernoulli : tube Venturi
# Section 1 : A1=0.01 m², v1=2 m/s, P1=P_atm, h=0
v1 = 2.0; A1 = 0.01; v2 = 6.0; A2 = A1*v1/v2    # continuité
P1 = P_atm
P2 = bernoulli(P1, v1, 0, rho_eau, P2=None, v2=v2, h2=0)
print(f"\nVenturi : v1={v1}m/s → v2={v2}m/s")
print(f"  P1 = {P1:.0f} Pa → P2 = {P2:.0f} Pa (dépression : {P1-P2:.0f} Pa)")

# Reynolds : tuyau diamètre 2cm, eau à 1 m/s
Re = reynolds(rho_eau, 1.0, 0.02, eta_eau)
print(f"\nReynolds tuyau D=2cm, v=1m/s : Re = {Re:.0f} → {'laminaire' if Re<2000 else 'turbulent'}")

```

## CONSTANTES UNIVERSELLES

|                                                        |                                              |
|--------------------------------------------------------|----------------------------------------------|
| $c = 3.000e8$ m/s                                      | Vitesse de la lumière dans le vide           |
| $G = 6.674e-11$ N·m <sup>2</sup> /kg <sup>2</sup>      | Constante de gravitation universelle         |
| $h = 6.626e-34$ J·s                                    | Constante de Planck                          |
| $\hbar = 1.055e-34$ J·s                                | $h/(2\pi)$                                   |
| $k_B = 1.381e-23$ J/K                                  | Constante de Boltzmann                       |
| $N_A = 6.022e23$ mol <sup>-1</sup>                     | Nombre d'Avogadro                            |
| $R = 8.314$ J/(mol·K)                                  | Constante des gaz parfaits = $k_B \cdot N_A$ |
| $e = 1.602e-19$ C                                      | Charge élémentaire                           |
| $m_e = 9.109e-31$ kg                                   | Masse de l'électron                          |
| $m_p = 1.673e-27$ kg                                   | Masse du proton                              |
| $\epsilon_0 = 8.854e-12$ F/m                           | Permittivité du vide                         |
| $\mu_0 = 4\pi \cdot 1e-7$ T·m/A                        | Perméabilité du vide                         |
| $\sigma = 5.670e-8$ W/(m <sup>2</sup> K <sup>4</sup> ) | Constante de Stefan-Boltzmann                |

## TABEAU DE BORD PAR BRANCHE

|                 |                                                                                              |
|-----------------|----------------------------------------------------------------------------------------------|
| Mécanique       | : $F=ma$ , $W=Fd$ , $E_c=(1/2)mv^2$ , $E_p=mgh$ , $p=mv$                                     |
| Thermodynamique | : $PV=nRT$ , $Q=mc\Delta T$ , $\Delta U=Q+W$ , $\eta_{\text{Carnot}}=1-T_f/T_c$              |
| Électrostatique | : $F=kq_1q_2/r^2$ , $E=kQ/r^2$ , $V=kQ/r$ , $U=qV$                                           |
| Circuits        | : $U=RI$ , $P=UI$ , $1/R_p=\sum 1/R_i$ , $Q=CU$                                              |
| Magnétisme      | : $F=qv \times B$ , $B_{\text{fil}}=\mu_0 I/(2\pi r)$ , $c=1/\sqrt{\mu_0 \epsilon_0}$        |
| Optique géom.   | : $n_1 \sin(t_1)=n_2 \sin(t_2)$ , $1/OA' - 1/OA = 1/f'$                                      |
| Ondes           | : $v=\lambda f$ , $f_{\text{Doppler}}=f \cdot v/(v \pm v_s)$ , $L=10 \cdot \log(I/I_0)$      |
| Quantique       | : $E=hf$ , $\lambda=h/p$ , $\Delta x \Delta p \geq \hbar/2$ , $E_n(H)=-13.6/n^2 \text{ eV}$  |
| Relativité      | : $\gamma=1/\sqrt{1-v^2/c^2}$ , $E=\gamma mc^2$ , $t'=\gamma t_0$                            |
| Stat. physique  | : $S=k_B \ln \Omega$ , $v_p=\sqrt{2k_B T/m}$ , Boltzmann: $\exp(-E/k_B T)$                   |
| Nucléaire       | : $N=N_0 \exp(-\lambda t)$ , $t_{1/2}=\ln(2)/\lambda$ , $E_{\text{liai}}=\Delta m \cdot c^2$ |
| Fluides         | : $P=F/A$ , $F_{\text{Arch}}=\rho g V$ , Bernoulli: $P+\rho v^2/2+\rho gh=\text{cte}$        |

#####

#

# CE QUE TU MAÎTRISES MAINTENANT :

#

- # ✓ Mécanique classique – cinématique, Newton, énergie, chocs
- # ✓ Thermodynamique – gaz parfait, 4 lois, entropie, Carnot
- # ✓ Électromagnétisme – Coulomb, circuits, Maxwell, Lorentz
- # ✓ Optique – Snell-Descartes, lentilles, interférences
- # ✓ Physique des ondes – paramètres, Doppler, acoustique
- # ✓ Mécanique quantique – De Broglie, Heisenberg, Schrödinger, H
- # ✓ Relativité – Lorentz,  $E=mc^2$ , contraction, dilatation
- # ✓ Physique statistique – Maxwell-Boltzmann, Boltzmann, entropie
- # ✓ Physique nucléaire – radioactivité, énergie de liaison, fission
- # ✓ Mécanique des fluides – Bernoulli, Archimède, viscosité, Reynolds

#

# PROCHAINES ÉTAPES NATURELLES :

- # → Physique de l'état solide (bandes d'énergie, semi-conducteurs)
- # → Physique des plasmas

```
#      → Cosmologie et physique des particules
#      → Électrodynamique quantique (QED)
#      → Mécanique lagrangienne et hamiltonienne
#
```

```
#####
```